

고급 자료구조 VITE 설계서

20203067 서현우

해당 과제는 아직 구현중에있어 계속하여 수정할것이며 계획또한 해당 계획서대로 되지않을 가능성 높음.

자료구조 : 기본적으로 gap buffer의 구조를 활용 할 것

```
typedef struct {  
  
    char* data;    // 텍스트 데이터를 저장할 동적 배열  
  
    size_t size;    // 현재 데이터 크기  
  
    size_t capacity; // 할당된 메모리 크기  
  
    size_t gapStart; // gap 시작 인덱스  
  
    size_t gapEnd;   // gap 끝 인덱스  
  
} GapBuffer;
```

gap buffer의 기본함수간단한 구현

// 갭 버퍼 초기화 함수

```
void initGapBuffer(GapBuffer* buffer) {  
    buffer->data = (char*)malloc(sizeof(char) * INITIAL_CAPACITY);  
    buffer->size = 0;  
    buffer->capacity = INITIAL_CAPACITY;  
    buffer->gapStart = 0;  
    buffer->gapEnd = INITIAL_CAPACITY - 1;  
}
```

// 갭 버퍼 내용 출력 함수

```
void printGapBuffer(const GapBuffer* buffer) {  
    for (size_t i = 0; i < buffer->size; ++i) {  
        if (i == buffer->gapStart) {  
            i = buffer->gapEnd + 1; // gap 건너뛰기  
        }  
        printf("%c", buffer->data[i]);  
    }  
    printf("\n");  
}
```

```
}
```

```
// 갭 버퍼에 문자 추가 함수
```

```
void insertChar(GapBuffer* buffer, char ch) {
```

```
    // gap이 비어있으면 확장
```

```
    if (buffer->gapStart == buffer->gapEnd) {
```

```
        size_t newCapacity = buffer->capacity * 2;
```

```
        buffer->data = (char*)realloc(buffer->data, sizeof(char) * newCapacity);
```

```
        buffer->gapEnd = newCapacity - (buffer->capacity - buffer->gapEnd);
```

```
        buffer->capacity = newCapacity;
```

```
    }
```

```
    // gap에 문자 추가
```

```
    buffer->data[buffer->gapStart] = ch;
```

```
    buffer->gapStart++;
```

```
    buffer->size++;
```

```
}
```

```
// 갭 버퍼에 문자열 추가 함수
```

```
void insertString(GapBuffer* buffer, const char* str) {
```

```
    size_t len = strlen(str);
```

```
    for (size_t i = 0; i < len; ++i) {
```

```
        insertChar(buffer, str[i]);
```

```
    }
```

```
}
```

```
// 갭 버퍼에서 문자 삭제 함수
```

```
void deleteChar(GapBuffer* buffer) {
```

```
    // gap 뒤에 있는 문자 삭제
```

```
    if (buffer->gapStart < buffer->size) {
```

```
        memmove(buffer->data + buffer->gapStart, buffer->data + buffer->gapStart + 1, buffer->size - buffer->gapStart);
```

```
        buffer->size--;
```

```
    }
```

```
}
```

```
// 갭 버퍼에서 문자열 삭제 함수
```

```
void deleteString(GapBuffer* buffer, size_t length) {
```

```
    // gap 뒤에 있는 문자열 삭제
```

```
    if (buffer->gapStart + length < buffer->size) {
```

```

        memmove(buffer->data + buffer->gapStart, buffer->data + buffer->gapStart + length,
buffer->size - buffer->gapStart - length);
        buffer->size -= length;
    }
}

// 갭 버퍼 해제 함수
void freeGapBuffer(GapBuffer* buffer) {
    free(buffer->data);
    buffer->size = 0;
    buffer->capacity = 0;
    buffer->gapStart = 0;
    buffer->gapEnd = 0;
}

```

gapbuffer를 통한 간단한 c언어에서의 구동 형태.

```

Current Text:
Enter command (i: insert, d: delete, q: quit): i
Enter text to insert: hello world!
Current Text: hello world!

Enter command (i: insert, d: delete, q: quit): i
Enter text to insert: hi! abcd
Current Text: hello world!
hi! abcd

```

1. 기본 구성

- 1) c언어 환경에서의 text editor 구현
- 2) 특정 키의 입력을 인식하기 위해 ncurses 라이브러리 활용
- 3) 버퍼의 활용
- 4) 파일의 open/close를 위한 fopen/fclose 함수 활용
 - 4-1)/vite의 명령을 입력받으면 fopen을 실행
 - 4-2)열린 텍스트를 읽고 버퍼에 읽히고 저장함

5)UI의 구현을 위해 `termios.h`의 헤더파일을 활용

6)탐색을 위한 KMP알고리즘을 구현

2. ncurses라이브러리 & 텍스트버퍼

1) 상태바, 메시지바

1. ncurses 라이브러리와 텍스트버퍼를 활용하여 파일이름, 커서위치 표시,

메시지바 도움말 표시를 수행

2.반전색상적용-ANSI_escape 활용 아래는 간단한 활용법

```
// ANSI escape codes

#define ANSI_RESET  "\x1b[0m"

#define ANSI_REVERSE "\x1b[7m"

// Function to print highlighted and reverse text for a specific row

void printHighlightedRow(const char *text, int highlightRow) {

    // ANSI escape code for reversing text

    printf(ANSI_REVERSE);

    // Split the text into rows

    int currentRow = 1;

    for (const char *ptr = text; *ptr; ++ptr) {

        putchar(*ptr);

        if (*ptr == '\n' && currentRow == highlightRow) {

            // If it's the target row, highlight it

            printf(ANSI_RESET ANSI_REVERSE);

        } else if (*ptr == '\n') {

            // If it's a different row, reset the color

            printf(ANSI_RESET);

            currentRow++;

        }

    }

}
```

```
// Reset the color at the end  
printf(ANSI_RESET);  
}
```

2) 텍스트 수정

1. ncurses라이브러리의 기본 기능활용, 백스페이스 및 delete등의 수정 수행.

3) 이동

ncurses의 기본기능활용.

1. →, ←

좌,우 화살표 키를 통해 왼쪽, 오른쪽으로 이동 및 행의 끝 또는 처음부분에서 해당 동작을 한 번 더 수행 할 경우 다음 행 또는 이전행으로 이동.

2. ↑, ↓

상,하 화살표 키를 통해 행의 상,하로 이동 및 텍스트 파일의 최상단, 최하단에 이동하였을 경우 더 이상 이동 할 수 없도록 수행

3. 특수 키

Key_상수와 getch 함수를 활용하여 해당키 작업 수행

-Page-up : 텍스트파일의 최상단으로 이동.

-Page-down : 텍스트파일의 최하단으로 이동

-Home : 현재 위치하고 있는 행의 처음으로 이동

-End : 현재 위치하고 있는 행의 마지막으로 이동

3. 저장

- 1) Ctrl+S의 입력을 감지하여 수행

- 2) 파일의 내용을 저장

2-1)파일의 이름이 지정되어 있지 않으면 이름을 입력 후 저장.

2-2)파일의 내용을 저장하였으면 file_saved변수를 true로 저장한다.

4. 탐색

1)Ctrl+F의 입력으로 수행

2)KMP 알고리즘을 구현, 알고리즘을 통한 문자열의 검색을 수행

3)검색 수행 후 enter입력 시 검색 종료 후 탐색결과에 커서 위치 하도록 구현

4)검색 수행 후 esc입력 시 검색 종료 후 탐색 이전의 커서 위치로 돌아감

5. 나가기

1)Ctrl+Q의 입력으로 수행

2)저장하지 않은 상태로 나가기를 원하면 "Ctrl+Q를 두 번 더 누르세요." 메시지 출력 후 해당 명령을 수행한다.

2-1)파일의 저장상태를 추적하기 위하여 file_saved변수를 활용하여 true면 한번의 입력으로 나가기를 수행, false면 두번의 입력을 더하여 나가기를 수행한다.

6. 메모리

1)효율적인 메모리 관리를 위해 텍스트 파일 길이만큼의 malloc을 통한 할당을 수행

2)프로그램이 종료되면 free를 통한 메모리의 해제를 수행

그 외 : 각 운영체제별로 구동가능한 함수 및 ascii코드 값, ncurses와 pdcurses등 운영체제별로 기능과 고유값이 다른 것들이 여럿 있으므로 해당 부분은 #ifndef등의 활용을 통해 운영체제별 동작을 정의 할 것.